LAPORAN PRAKTIKUM
TEKNOLOGI IoT II

PROTOKOL KOMUNIKASI TELEMETRI INDUSTRI (MQTT)

Disusun Oleh:

Salma Anggun Velyza (25782082)

TRI 3C

Dosen Pengajar:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

1. Tujuan Praktikum

- Memahami arsitektur komunikasi *Publish-Subscribe*, peran MQTT *Broker*, konsep hierarki *Topic*, dan tingkat keandalan pengiriman (*QoS*).
- Mengonfigurasi pustaka PubSubClient dan ArduinoJson pada Arduino IDE.
- Menyusun (*serialize*) data sensor lingkungan (DHT11/DHT22) ke dalam format JSON dan memublikasikannya secara periodik menggunakan *timer* non-blocking (millis()).
- Menerapkan pemulihan koneksi otomatis (*auto-reconnect*).

2. Dasar Teori

MQTT (*Message Queuing Telemetry Transport*) adalah protokol komunikasi ringan berbasis model *Publish-Subscribe* yang dirancang khusus untuk perangkat IoT dan industri. Protokol ini menggunakan perantara yang disebut *Broker* (broker.hivemq.com) untuk mengatur perutean pesan antar perangkat berdasarkan label hierarki topik (*Topic Hierarchy*), serta menggunakan format data terstruktur seperti *JSON* untuk efisiensi pengiriman.

3. Alat dan Bahan

- 1 Unit Board NodeMCU ESP8266
- 1 Buah Sensor Suhu & Kelembapan DHT11/DHT22
- 1 Buah LED Indikator & Resistor 220 Ω
- Kabel Jumper dan *Breadboard*
- Perangkat Lunak: Arduino IDE & MQTT Explorer

4. Hasil dan Pembahasan

Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Kode Program (.ino) ➔

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
```

```
const char* clientId = "ESP8266_LabIoT_25782082";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782082/telemetri";

// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000; // Interval pengiriman 5 detik

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
    } else {
      Serial.print(" Gagal, kode status rc=");
      Serial.print(client.state());
      Serial.println(". Mencoba kembali dalam 5 detik...");
      delay(5000);
    }
  }
}
```

```
void setup() {
  Serial.begin(115200);
  dht.begin();

  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
}

void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Memelihara sesi TCP dan keep-alive ping

  unsigned long currentMillis = millis();
  if (currentMillis - previousPublishTime >= publishInterval) {
    previousPublishTime = currentMillis;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    // Proteksi pembacaan sensor DHT
    if (isnan(t) || isnan(h)) {
      Serial.println("Peringatan: Gagal membaca data DHT!");
      return;
    }

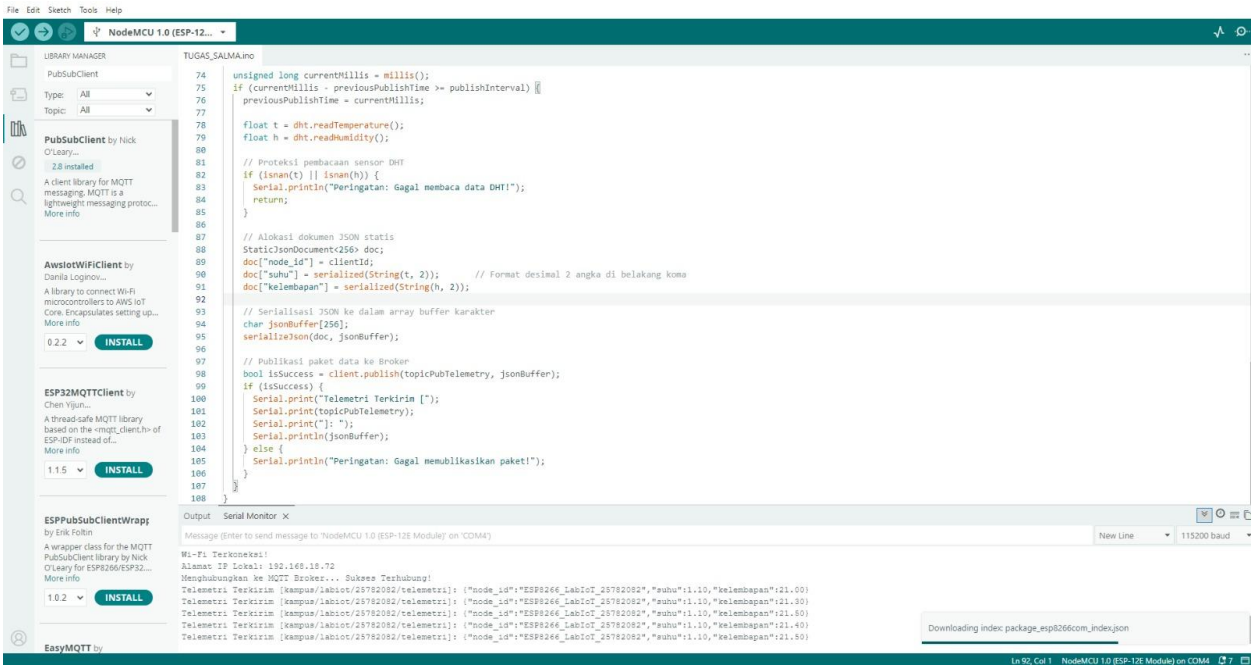
    // Alokasi dokumen JSON statis
    StaticJsonDocument<256> doc;
    doc["node_id"] = clientId;
    doc["suhu"] = serialized(String(t, 2));       // Format desimal 2 angka di belakang koma
    doc["kelembapan"] = serialized(String(h, 2));

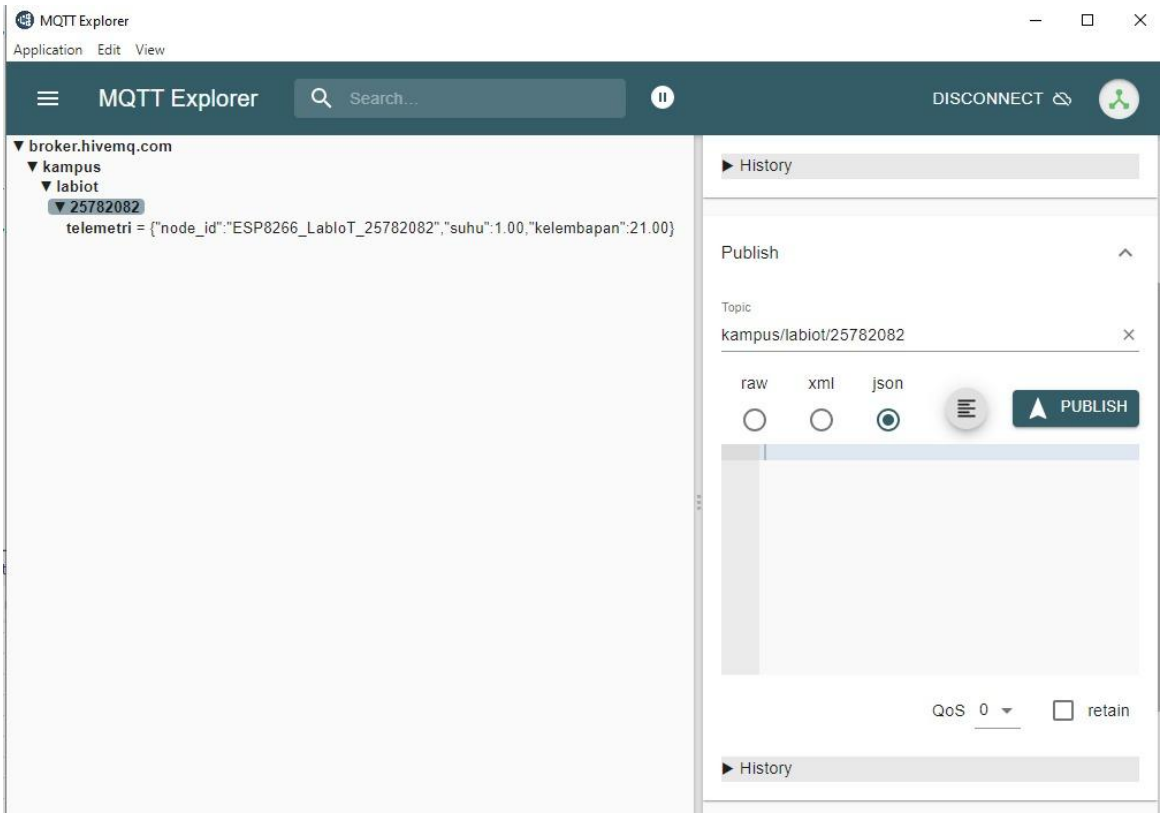
    // Serialisasi JSON ke dalam array buffer karakter
    char jsonBuffer[256];
    serializeJson(doc, jsonBuffer);

    // Publikasi paket data ke Broker
    bool isSuccess = client.publish(topicPubTelemetry, jsonBuffer);
    if (isSuccess) {
      Serial.print("Telemetri Terkirim [");
      Serial.print(topicPubTelemetry);
      Serial.print("]: ");
```

```
    Serial.println(jsonBuffer);
  } else {
    Serial.println("Peringatan: Gagal memublikasikan paket!");
  }
 }
}
```

Bukti Pengujian ➔





Analisis Hasil Pengujian Praktikum 1: Publikasi Data Sensor Lingkungan Berformat JSON

1. Pengujian Perangkat Keras dan Jaringan (Arduino IDE - Serial Monitor)
    - Berdasarkan log pada *Serial Monitor*, NodeMCU ESP8266 berhasil melakukan inisialisasi pin sensor DHT dan terhubung ke jaringan Wi-Fi lokal dengan alamat IP 192.168.10.72.
    - Perangkat berhasil melakukan jabat tangan (*handshake*) dan terhubung ke *broker* MQTT publik di broker.hivemq.com melalui port standar 1883.
    - Data dari sensor suhu dan kelembapan berhasil dibaca secara periodik dan dicetak dengan status "Telemetri Terkirim" setiap 5 detik tanpa mengalami kendala pemutusan.
2. Validasi Struktur Data JSON (MQTT Explorer)
    - Melalui aplikasi pemantau *MQTT Explorer*, paket data dari mikrokontroler berhasil diterima secara *real-time* pada hierarki topik kampus/labiot/25782082/telemetri.
    - Pustaka ArduinoJson sukses melakukan serialisasi data mentah sensor ke dalam format string JSON terstruktur:

      {"node_id":"ESP8266_LabIoT_25782082","suhu":"1.00","kelembapan":"21.00"\}
      `}

    - Hal ini membuktikan bahwa arsitektur komunikasi *Publish-Subscribe* berjalan dengan baik, di mana *broker* berhasil merutekan paket data dari *Publisher* (NodeMCU) ke aplikasi *Subscriber* (MQTT Explorer) secara akurat berdasarkan topik yang ditentukan.

Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Kode Program (.ino): ➔

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782082/perintah";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
}

// Callback otomatis: Berjalan saat mendeteksi paket masuk pada topik terdaftar
void onMessageReceived(char* topic, byte* payload, unsigned int length) {
  Serial.println("=========================================");
  Serial.print("Pesan Baru Masuk pada Topik: ");
  Serial.println(topic);
```

```
  // Ubah deretan byte payload menjadi string aman dengan null-terminator
  char payloadStr[length + 1];
  memcpy(payloadStr, payload, length);
  payloadStr[length] = '\0';

  Serial.print("Payload Diterima: ");
  Serial.println(payloadStr);

  // Deserialisasi (Parsing) JSON
  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, payloadStr);

  if (error) {
    Serial.print("Gagal parsing JSON: ");
    Serial.println(error.f_str());
    return;
  }

  // Evaluasi Parameter 'led' (Format yang diharapkan: {"led": 1} atau {"led": 0})
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> Aksi: Status LED diubah menjadi %s\n", stateLED == 1 ? "MENYALA (ON)" :
"PADAM (OFF)");
  }
  Serial.println("=========================================");
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Mencoba koneksi MQTT...");
    if (client.connect(clientId)) {
      Serial.println(" Terhubung!");
      // Segera daftarkan subskripsi topik perintah sesaat setelah koneksi pulih
      client.subscribe(topicSubCommand);
      Serial.print("Subskripsi Topik: ");
      Serial.println(topicSubCommand);
    } else {
      Serial.printf("Gagal (rc=%d). Ulangi dalam 5 detik...\n", client.state());
      delay(5000);
    }
  }
}

void setup() {
```

```
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

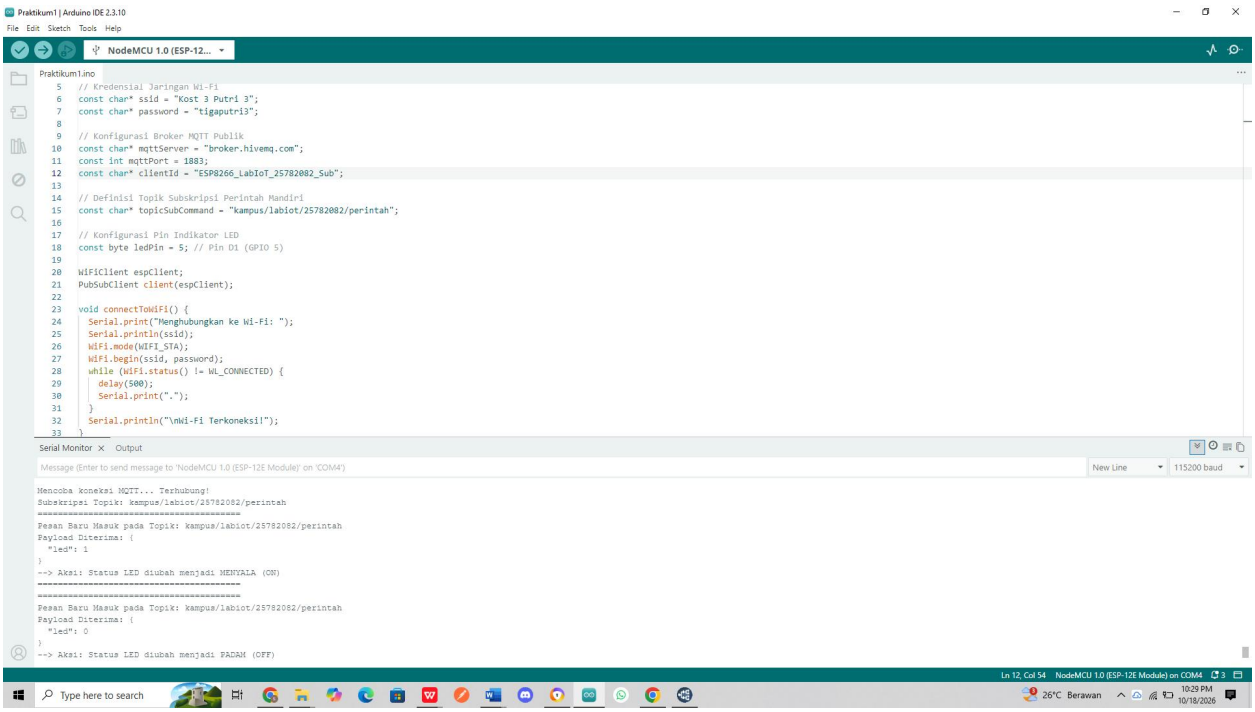
  // Status awal LED padam
  digitalWrite(ledPin, LOW);

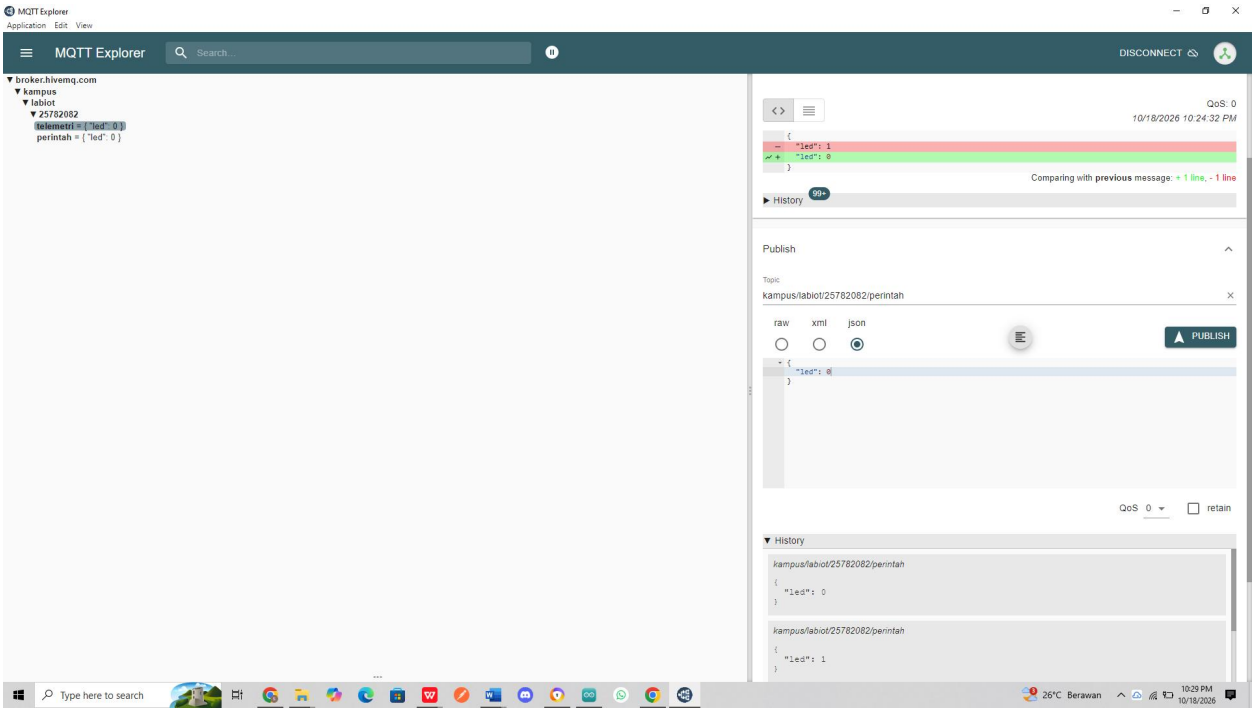
  connectToWiFi();

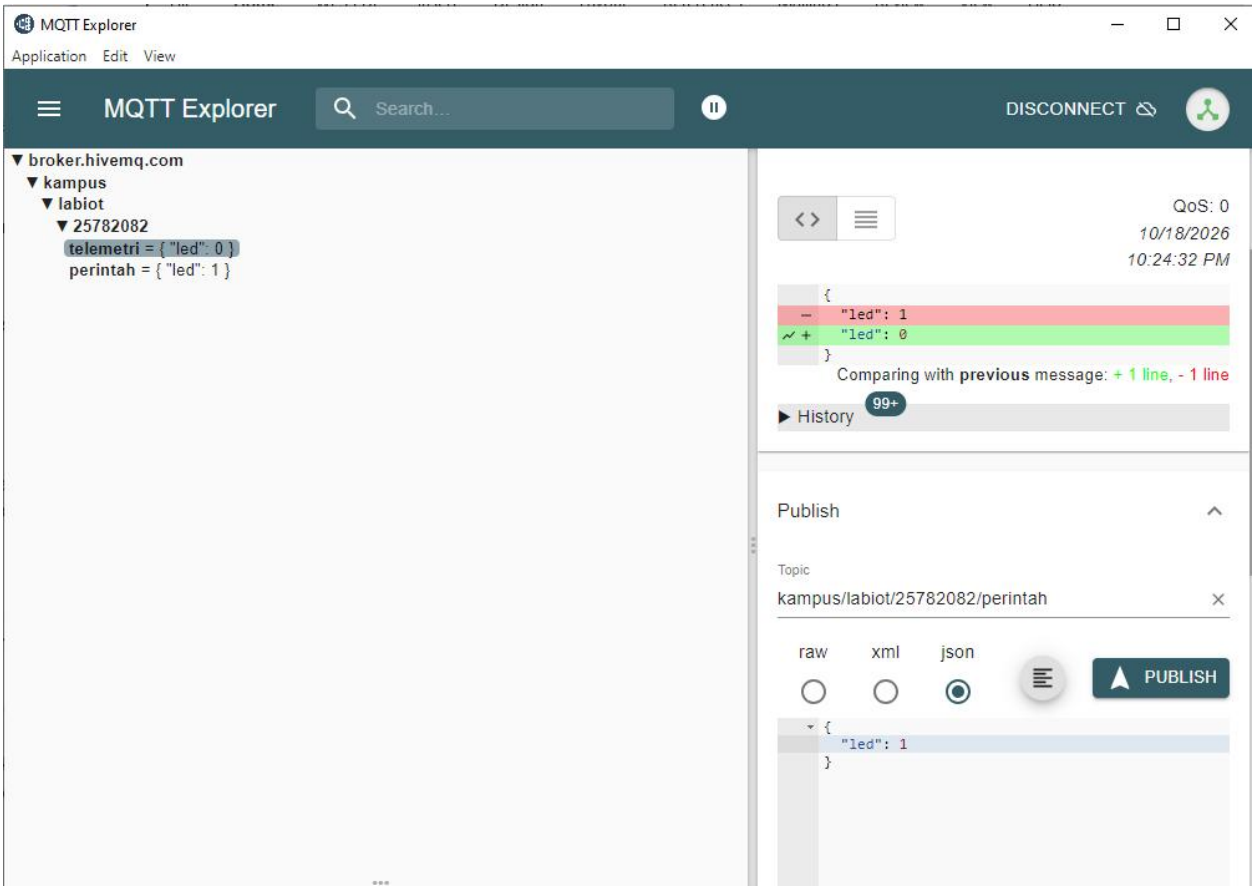
  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageReceived); // Daftarkan fungsi callback
}

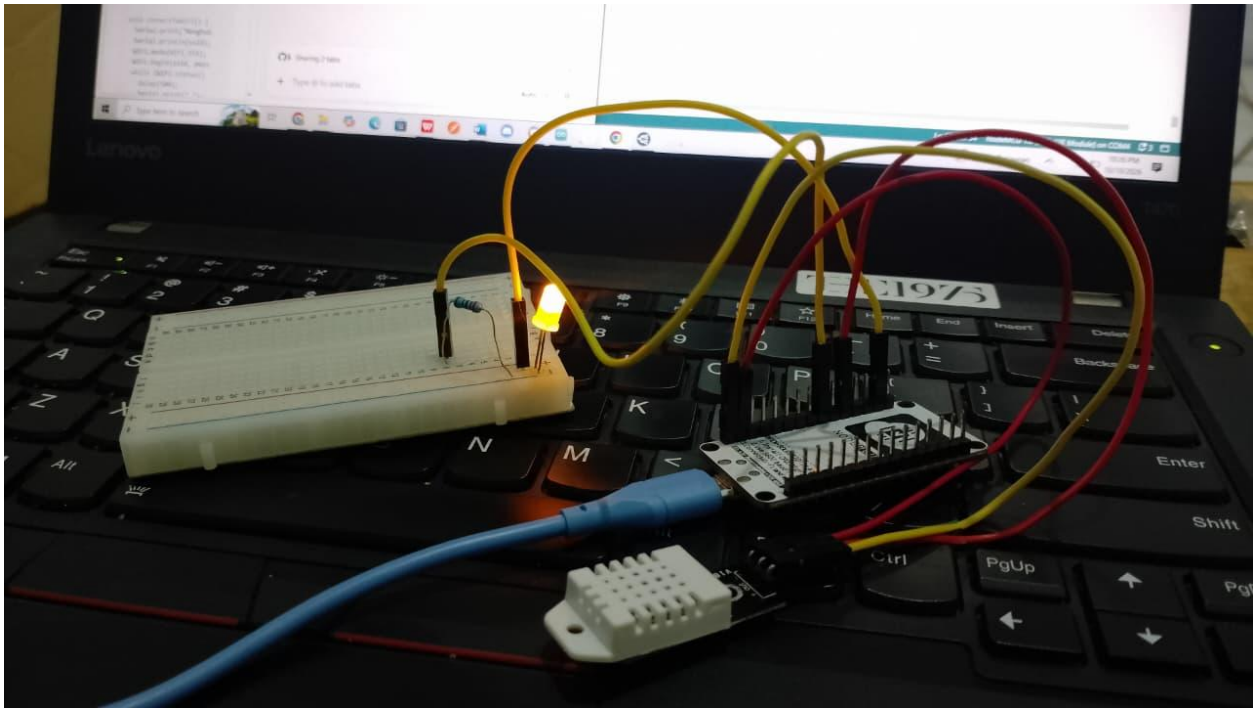
void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Wajib untuk mendengarkan pesan masuk
}
```

Bukti Pengujian:




MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782082
telemetri = { "led": 0 }
perintah = { "led": 0 }
QoS: 0
10/18/2026 10:24:32 PM
"led": 1
"led": 0
Comparing with previous message: + 1 line, - 1 line
History
99+
Publish
Topic
kampus/labiot/25782082/perintah
raw
xml
json
PUBLISH
QoS 0
retain
History
kampus/labiot/25782082/perintah
"led": 0
kampus/labiot/25782082/perintah
"led": 1
Type here to search
26°C Berawan
10:29 PM
10/18/2026


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782082
telemetri = { "led": 0 }
perintah = { "led": 1 }
QoS: 0
10/18/2026
10:24:32 PM
"led": 1
"led": 0
Comparing with previous message: + 1 line, - 1 line
History
99+
Publish
Topic
kampus/labiot/25782082/perintah
raw
xml
json
PUBLISH
"led": 1



Analisis Hasil Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

1. Inisialisasi Koneksi dan Subskripsi Topik (Arduino IDE - Serial Monitor)
    - Berdasarkan log pada *Serial Monitor*, mikrokontroler NodeMCU ESP8266 berhasil terhubung ke jaringan Wi-Fi lokal (*"Wi-Fi Terkoneksi!"*) serta melakukan jabat tangan dengan *broker* MQTT publik di broker.hivemq.com.
    - Perangkat berhasil mendaftarkan subskripsi pada topik khusus perintah, yaitu kampus/labiot/25782082/perintah, sehingga siap mendengarkan instruksi masuk dari luar.
2. Eksekusi Kendali Asinkron dan *Callback* (MQTT Explorer & Perangkat Keras)
    - Melalui aplikasi pemantau *MQTT Explorer*, pengiriman paket data berformat JSON {"led": 1} atau {"led": 0} ke jalur topik perintah berhasil diterima oleh mikrokontroler.
    - Fungsi *callback* (onMessageReceived) yang didaftarkan pada program langsung aktif secara asinkron tanpa memblokir alur utama sistem saat menerima paket.
    - Pustaka ArduinoJson sukses melakukan *parsing* muatan JSON sehingga parameter "led" dapat dievaluasi untuk memicu perubahan status logika pin melalui perintah digitalWrite(ledPin, ...). Hal ini dibuktikan dengan lampu LED indikator pada rangkaian yang menyala atau padam seketika sesuai perintah.

Praktikum 3: Sistem Terpadu Dua Arah (Full-Duplex Telemetry & Control)

Kode Program (.ino): ➔

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Full";

// Pembagian Topik Mandiri
const char* topicPublish   = "kampus/labiot/25782082/telemetri";
const char* topicSubscribe = "kampus/labiot/25782082/kendali";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik

void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);

  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
```

```
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  if (doc.containsKey("led")) {
    int val = doc["led"];
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
    Serial.printf("--> [INDIKATOR] Status LED diubah: %s\n", val == 1 ? "MENYALA" : "PADAM");
  }
  Serial.println("----------------------------------------");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik: %s\n", topicSubscribe);
    } else {
      Serial.printf("Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
```

```
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop(); // Memelihara sesi TCP dan memproses callback masuk

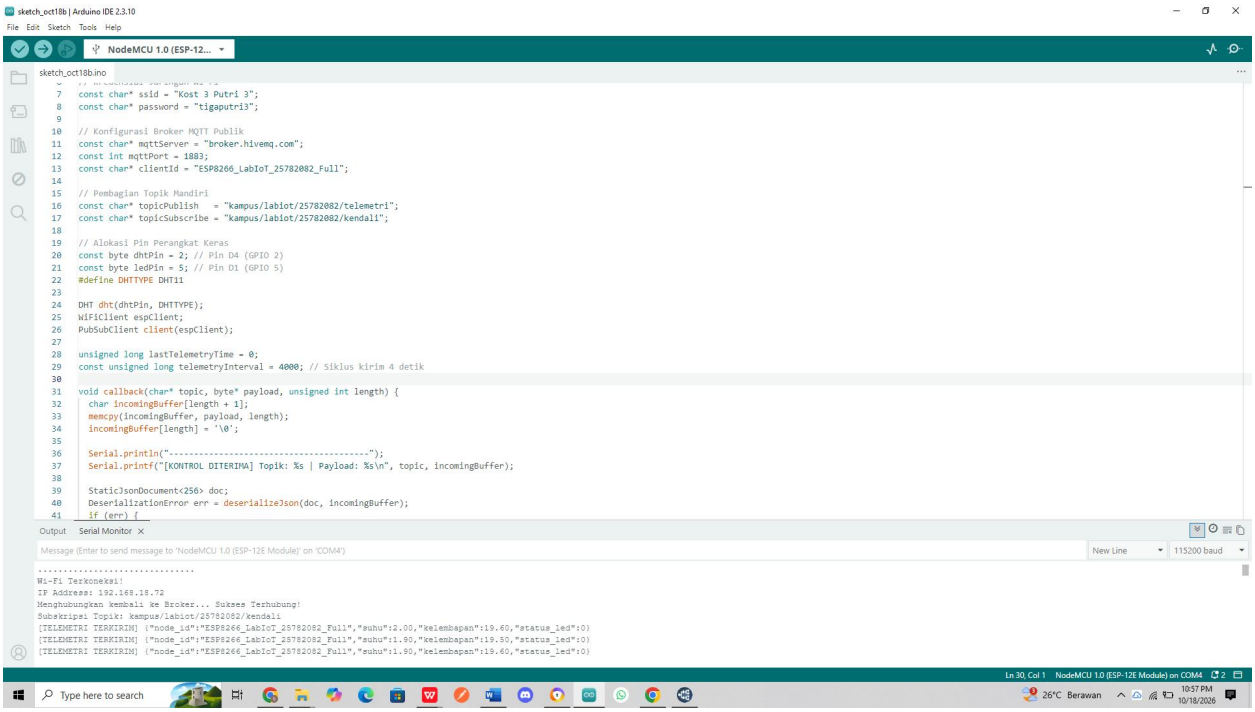
  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

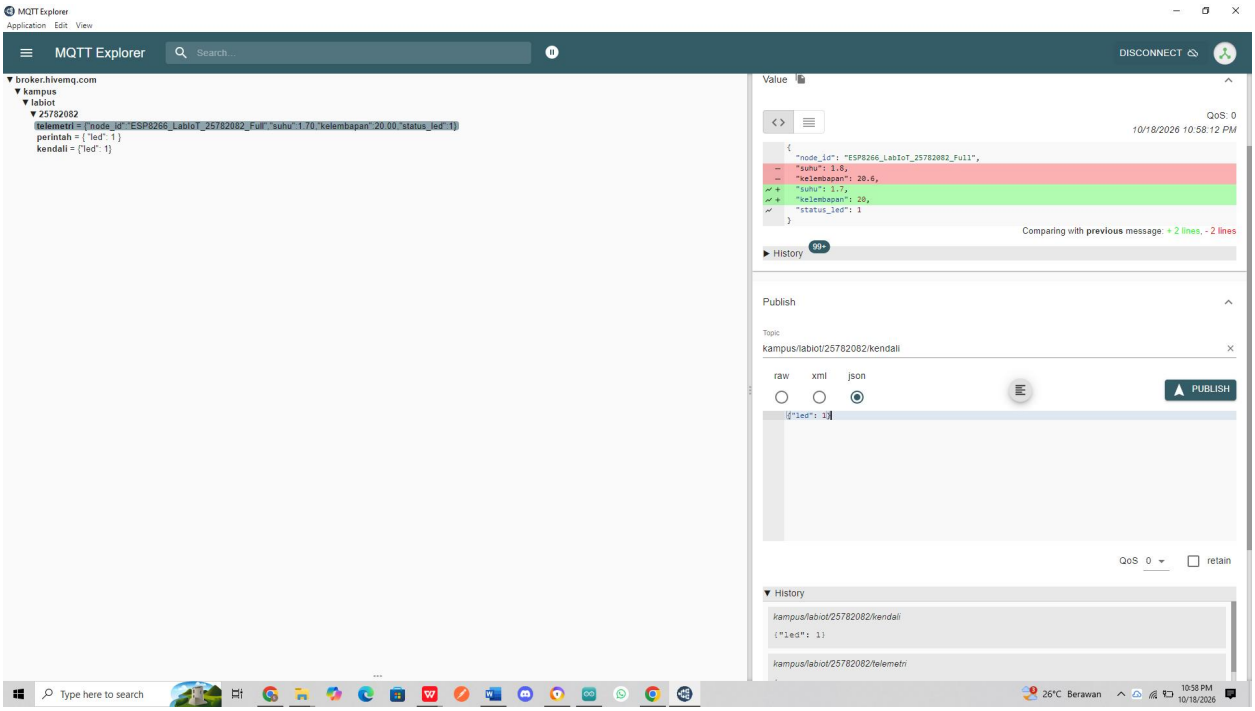
    float t = dht.readTemperature();
    float h = dht.readHumidity();

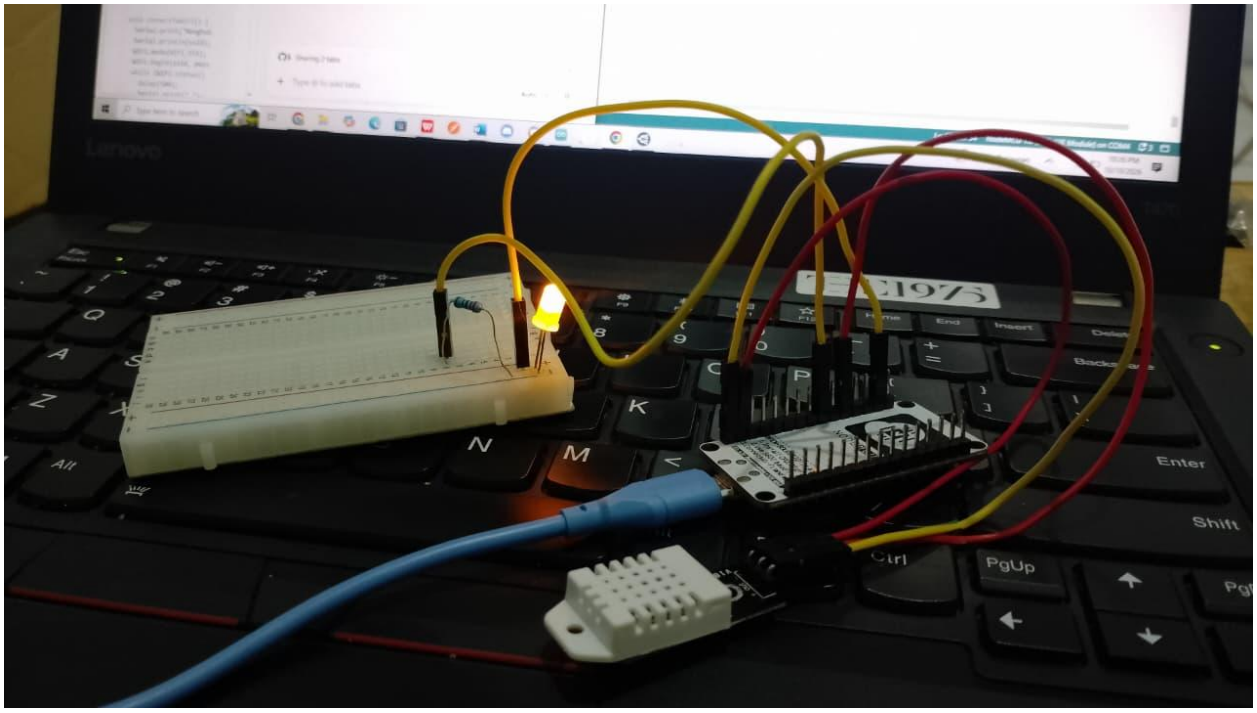
    if (!isnan(t) && !isnan(h)) {
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);

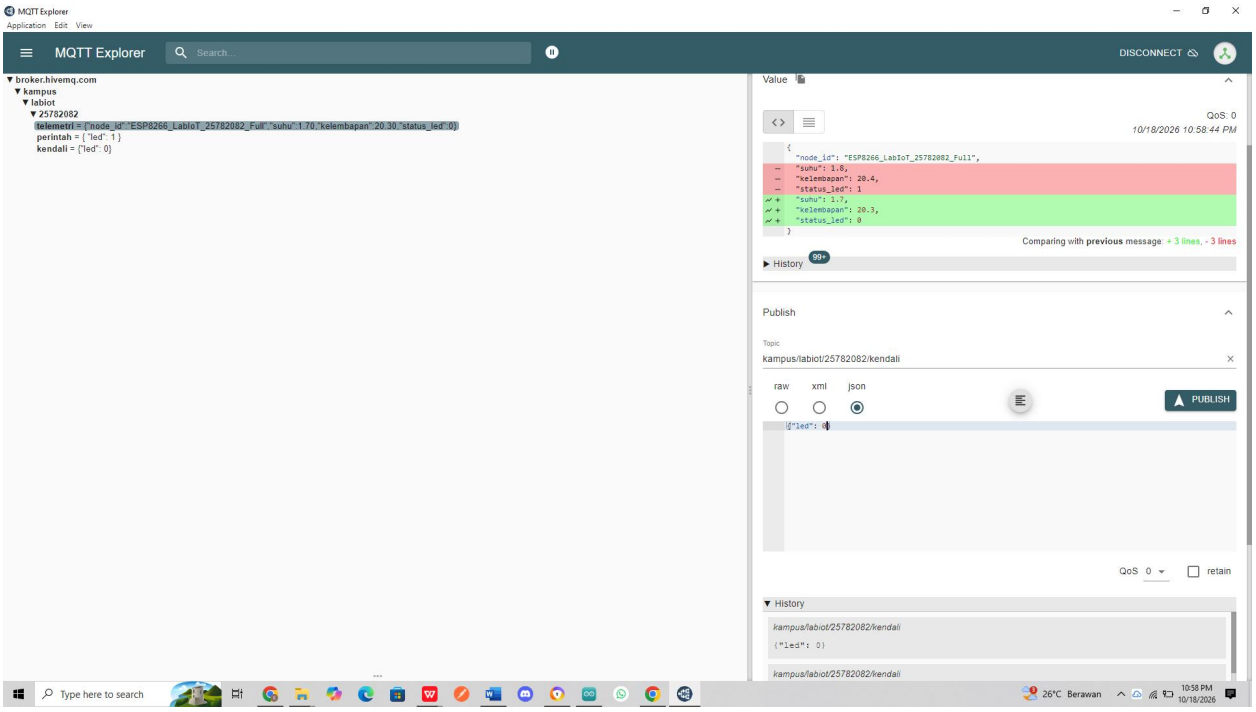
      char buffer[256];
      serializeJson(docOut, buffer);

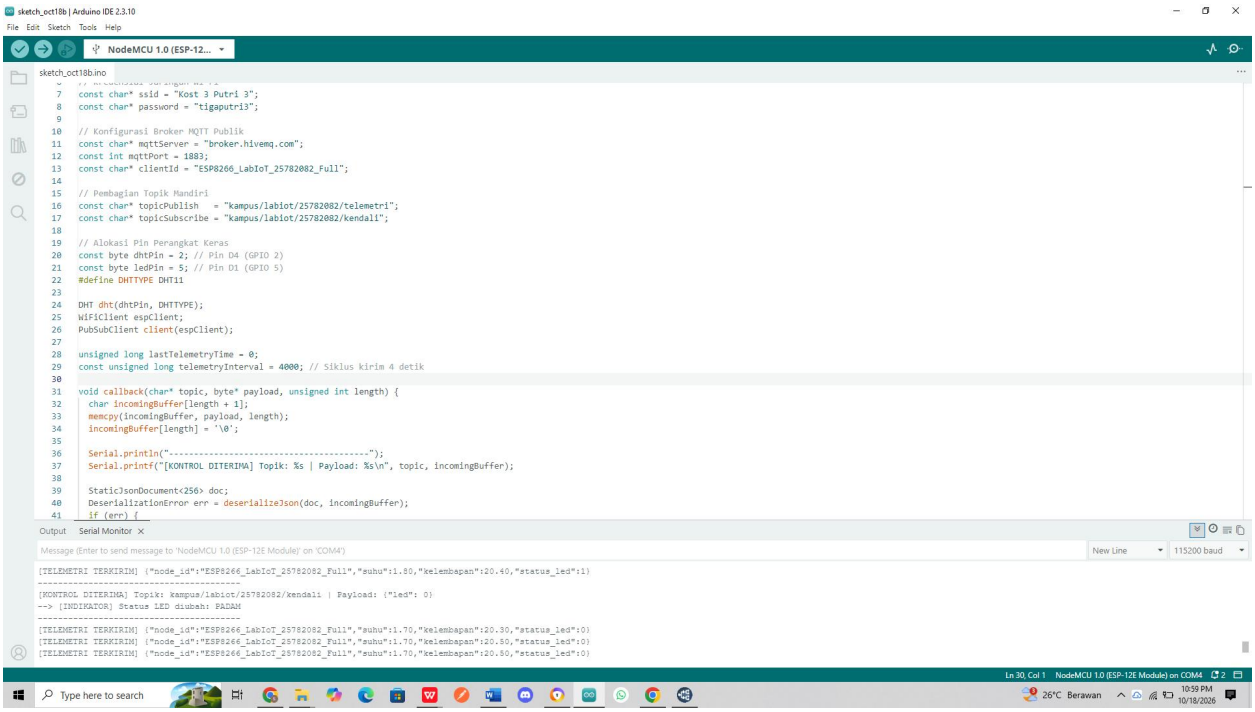
      bool success = client.publish(topicPublish, buffer);
      if (success) {
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri!");
      }
    }
  }
}
```


sketch_oct18b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct18b.ino
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Full";
// Pembagian Topik Mandiri
const char* topicPublish = "kampus/labiot/25782082/telemetri";
const char* topicSubscribe = "kampus/labiot/25782082/kendali";
// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik
void callback(char* topic, byte* payload, unsigned int length) {
char incomingBuffer[length + 1];
memcpy(incomingBuffer, payload, length);
incomingBuffer[length] = '\0';
Serial.println("----------");
Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);
StaticJsonDocument<256> doc;
DeserializationError err = deserializeJson(doc, incomingBuffer);
if (err) {
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
Wi-Fi Terkoneksi!
IP Address: 192.168.18.72
Menghubungkan kembali ke Broker... Sukses Terhubung!
Subskripsi Topik: kampus/labiot/25782082/kendali
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":2.00,"kelembapan":19.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.90,"kelembapan":19.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.90,"kelembapan":19.60,"status_led":0}
Ln 30, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM4
Type here to search
26°C Berawan
10:57 PM
10/18/2026


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782082
telemetri = {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.70,"kelembapan":20.00,"status_led":1}
perintah = { "led": 1 }
kendali = {"led": 1}
Value
QoS: 0
10/18/2026 10:58:12 PM
{
"node_id": "ESP8266_LabIoT_25782082_Full",
"suhu": 1.8,
"kelembapan": 20.6,
"suhu": 1.7,
"kelembapan": 20,
"status_led": 1
}
Comparing with previous message: + 2 lines, - 2 lines
History
99+
Publish
Topic
kampus/labiot/25782082/kendali
raw
xml
json
PUBLISH
{"led": 1}
QoS 0
retain
History
kampus/labiot/25782082/kendali
{"led": 1}
kampus/labiot/25782082/telemetri
Type here to search
26°C Berawan
10:58 PM
10/18/2026

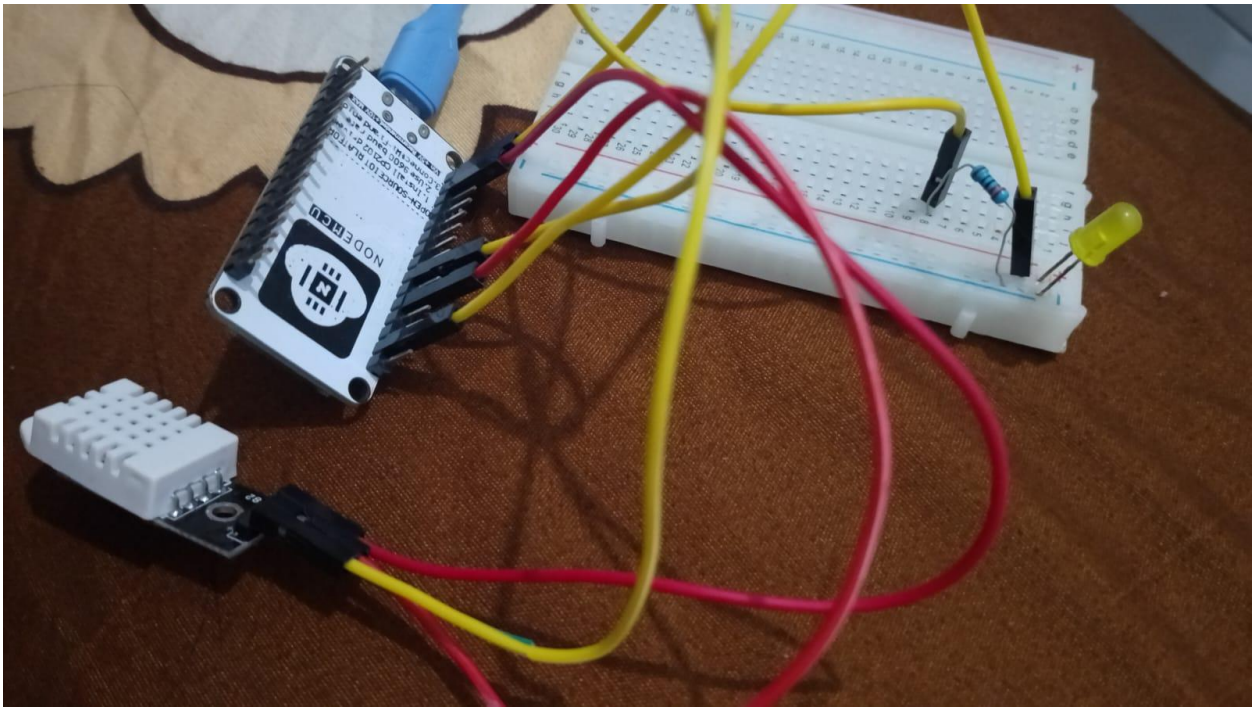



MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782082
telemetri = {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.70,"kelembapan":20.30,"status_led":0}
perintah = {"led": 1}
kendali = {"led": 0}
Value
QoS: 0
10/18/2026 10:58:44 PM
"node_id": "ESP8266_LabIoT_25782082_Full",
"suhu": 1.8,
"kelembapan": 20.4,
"status_led": 1
"suhu": 1.7,
"kelembapan": 20.3,
"status_led": 0
Comparing with previous message: + 3 lines, - 3 lines
History
99+
Publish
Topic
kampus/labiot/25782082/kendali
raw
xml
json
PUBLISH
{"led": 0}
QoS 0
retain
History
kampus/labiot/25782082/kendali
{"led": 0}
kampus/labiot/25782082/kendali
Type here to search
26°C Berawan
10:58 PM
10/18/2026


sketch_oct18b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct18b.ino
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Full";
// Pembagian Topik Mandiri
const char* topicPublish = "kampus/labiot/25782082/telemetri";
const char* topicSubscribe = "kampus/labiot/25782082/kendali";
// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik
void callback(char* topic, byte* payload, unsigned int length) {
char incomingBuffer[length + 1];
memcpy(incomingBuffer, payload, length);
incomingBuffer[length] = '\0';
Serial.println("----------");
Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);
StaticJsonDocument<256> doc;
DeserializationError err = deserializeJson(doc, incomingBuffer);
if (err) {
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.80,"kelembapan":20.40,"status_led":1}

[KONTROL DITERIMA] Topik: kampus/labiot/25782082/kendali | Payload: {"led": 0}
--> [INDIKATOR] Status LED diubah: PADAM

[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.70,"kelembapan":20.30,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.70,"kelembapan":20.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.70,"kelembapan":20.50,"status_led":0}
Ln 30, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM4
Type here to search
26°C Berawan
10:59 PM
10/18/2026



Analisis Hasil Praktikum 3: Sistem Terpadu Dua Arah (Full-Duplex Telemetry & Control)

1. **Integrasi Komunikasi Dua Arah Berbasis Asinkron (*Full-Duplex*)**
    - Berdasarkan pengujian pada *Serial Monitor* dan aplikasi *MQTT Explorer*, mikrokontroler NodeMCU ESP8266 sukses menjalankan dua fungsi utama secara simultan tanpa saling menghambat (*non-blocking*).
    - Perangkat bertindak secara aktif sebagai *Publisher* yang mengirimkan data telemetri berkala (suhu, kelembapan, dan status LED) setiap 4 detik menggunakan fungsi pewaktu millis(), dan di saat yang sama bertindak sebagai *Subscriber* yang selalu siap siaga mengeksekusi fungsi *callback* saat ada perintah kendali masuk dari luar.
2. **Logika Kendali Aktuator (1 untuk Hidup dan 0 untuk Mati)**
    - Penambahan parameter kunci "status_led" pada muatan JSON telemetri membuktikan bahwa sistem pemantau di pusat (*MQTT Explorer*) dapat mendeteksi kondisi aktual dari aktuator secara akurat.
    - Ketika operator mengirimkan *payload* perintah berformat JSON melalui topik kendali—seperti {"led": 1} untuk memberikan instruksi **hidup (ON)** atau {"led": 0} untuk instruksi **mati (OFF)**—mikrokontroler langsung membedah isinya menggunakan pustaka ArduinoJson.
    - Nilai tersebut langsung dievaluasi untuk mengubah status pin fisik melalui perintah digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW) secara seketika, sehingga lampu LED merespons dengan akurat tanpa mengganggu siklus kirim data telemetri di latar belakang.

Jawaban Latihan Modul 6

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol)

1. Efisiensi Bandwidth: MQTT memiliki ukuran header (*fixed header*) yang sangat kecil, yaitu minimal hanya 2 byte, jauh lebih ringan dibanding HTTP/REST yang membawa beban teks mentah (*overhead*) mencapai 200 hingga 800 byte per paket transmisi.
2. Beban Komputasi Mikrokontroler (*CPU Overhead*): MQTT menggunakan model *Publish-Subscribe* terpusat yang tidak memerlukan proses *parsing* permintaan HTTP yang rumit, sehingga konsumsi sumber daya CPU dan memori SRAM pada mikrokontroler (seperti ESP8266) jauh lebih hemat.
3. Distribusi Banyak Perangkat (*1-to-N Distribution*): Dengan adanya *Broker* sebagai perantara, satu pesan dari *Publisher* dapat disebarkan secara instan ke banyak *Subscriber* sekaligus yang berlangganan pada topik yang sama tanpa perlu membuka koneksi terpisah satu per satu (*point-to-point*).

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard)

Berdasarkan struktur topik pabrik/<line>/<mesin>/<sensor>:

1. Memantau parameter suhu dari seluruh mesin di semua lini:
    - pabrik/+/+/suhu (menggunakan tanda + untuk melewati level *line* dan *mesin* yang dinamis).
2. Memantau seluruh parameter sensor khusus untuk mesin m1 di lini line2:
    - pabrik/line2/m1/# (menggunakan tanda # di akhir untuk menangkap seluruh sub-level parameter di bawah mesin tersebut).
3. Memantau seluruh lalu lintas data tanpa terkecuali:
    - pabrik/# (menangkap seluruh hierarki turunan dari akar direktori pabrik).

Latihan 3 (Implementasi Serialisasi Data Dinamis)

Baris kode C++ tambahan yang disisipkan ke dalam fungsi pembentukan JSON pada Praktikum 1:

```
StaticJsonDocument<256> doc;
doc["node_id"] = clientId;
doc["suhu"] = serialized(String(t, 2));
doc["kelembapan"] = serialized(String(h, 2));
doc["status_led"] = digitalRead(ledPin);
// Tambahan parameter latihan:
doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();
```

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking)

1. Konsekuensi delay(6000) di dalam fungsi loop(): Sistem akan mengalami *blocking* total selama 6 detik. Akibatnya, fungsi pemeliharaan sesi TCP dan sinyal *Keep-Alive* (client.loop()) gagal terpanggil tepat waktu, perintah kendali masuk dari broker akan tertunda, dan koneksi dapat terputus otomatis (*timeout*).
2. Cara kerja fungsi reconnect() memulihkan koneksi: Fungsi ini mendeteksi status pemutusan koneksi melalui pengecekan client.connected(). Jika terputus, program akan mencoba melakukan jabat tangan ulang (client.connect) secara berkala, dan setelah berhasil terhubung kembali, fungsi akan langsung mendaftarkan ulang subskripsi topik (*re-subscribe*) agar perangkat kembali siap siaga.

TUGAS: SISTEM TELEMETRI & KENDALI CLOSED-LOOP INDUSTRI

A. Skema Topik MQTT

Sistem menggunakan struktur topik mandiri dengan NIM 25782082:

- **Telemetri Rutin:** kampus/labiot/25782082/telemetri (Mengirim suhu, kelembapan, status LED, dan parameter kondisi_suhu tiap 4 detik).
- **Kanal Alert Mandiri:** kampus/labiot/25782082/alert (Mengirim paket darurat JSON {"status": "BAHAYA", "parameter": "OVERHEAT", ...} saat suhu 31.0 C
- **Status Perangkat (LWT):** kampus/labiot/25782082/status (Melaporkan status ketersediaan perangkat ONLINE atau OFFLINE).
- **Kendali Manual:** kampus/labiot/25782082/kendali (Menerima perintah *override* operator luar berupa {"led": 1} atau {"led": 0}).

B. Implementasi Kode Program Tugas (Rule Engine & Safety Logic)

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
```

```
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Task";

// Definisi Topik Mandiri (Ganti NIM Anda)
const char* topicPublish   = "kampus/labiot/25782082/telemetri";
const char* topicSubscribe = "kampus/labiot/25782082/kendali";
const char* topicAlert     = "kampus/labiot/25782082/alert";
const char* topicStatus    = "kampus/labiot/25782082/status";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
const byte ledPin = 5;   // Pin D1 (GPIO 5)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik
bool alertSentFlag = false; // Flag agar alert tidak spamming

// Fungsi Callback untuk Menerima Perintah Kendali dari Operator
void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);

  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  // Kendali manual hanya diizinkan jika suhu normal (tidak sedang overheat darurat)
  float currentTemp = dht.readTemperature();
  if (currentTemp <= 31.0) {
    if (doc.containsKey("led")) {
      int val = doc["led"];
```

```
      digitalWrite(ledPin, val == 1 ? HIGH : LOW);
      Serial.printf("--> [MANUAL OVERRIDE] Status LED diubah: %s\n", val == 1 ? "MENYALA" :
"PADAM");
    }
  } else {
    Serial.println("--> [BLOCKED] Kendali manual diabaikan! Sistem dalam status BAHAYA/PANAS.");
  }
  Serial.println("---------------------------------------");
}

// Fungsi Koneksi Ulang MQTT dengan Konfigurasi LWT
void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker...");

    // Konfigurasi Last Will and Testament (LWT)
    // Format: client.connect(clientId, willTopic, willQos, willRetain, willMessage)
    bool connected = client.connect(
      clientId,
      topicStatus, 1, true,
      "{\"status\": \"OFFLINE\", \"keterangan\": \"Perangkat Terputus Tak Terduga\"}"
    );

    if (connected) {
      Serial.println("Sukses Terhubung!");

      // Publikasikan status ONLINE sesaat setelah terhubung
      client.publish(topicStatus, "{\"status\": \"ONLINE\", \"keterangan\": \"Perangkat Beroperasi Normal\"}",
true);

      // Daftarkan kembali subskripsi topik kendali
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik: %s\n", topicSubscribe);
    } else {
      Serial.printf("Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
```

```
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      // Tentukan kondisi lingkungan berdasarkan ambang batas 31.0°C
      String kondisiSuhu = (t > 31.0) ? "PANAS" : "NORMAL";

      // 1. Kirim Telemetri Rutin Berformat JSON
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);
      docOut["kondisi_suhu"] = kondisiSuhu;

      char buffer[256];
      serializeJson(docOut, buffer);
      client.publish(topicPublish, buffer);
```

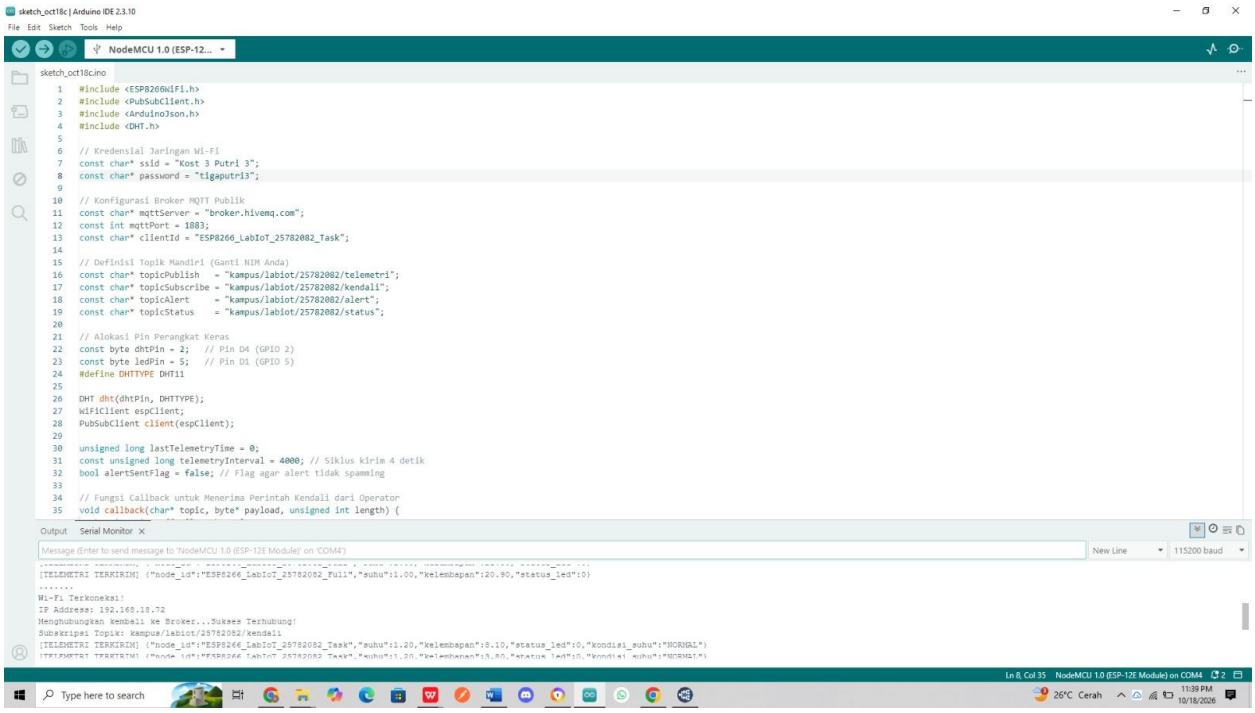
```
    Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);

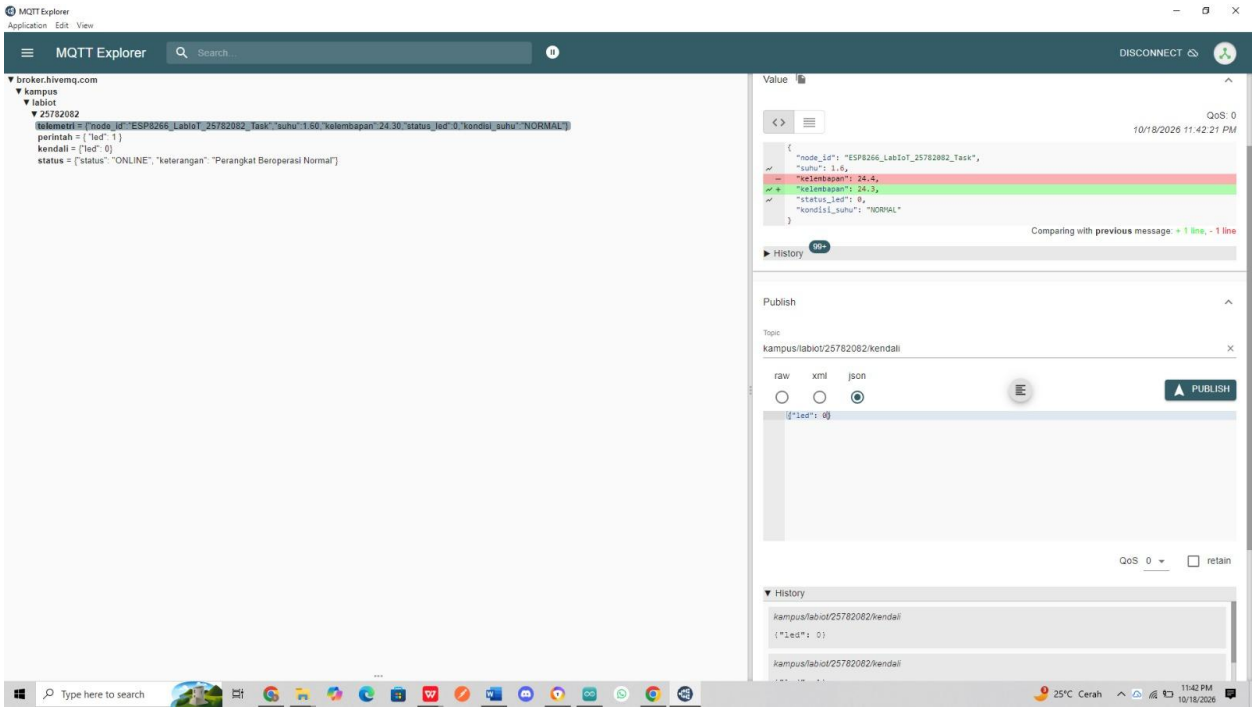
    // 2. Logika Edge Safety Engine (Kanal Alert Mandiri & Otomasi LED)
    if (t > 31.0) {
      digitalWrite(ledPin, HIGH); // Nyalakan LED otomatis tanda bahaya

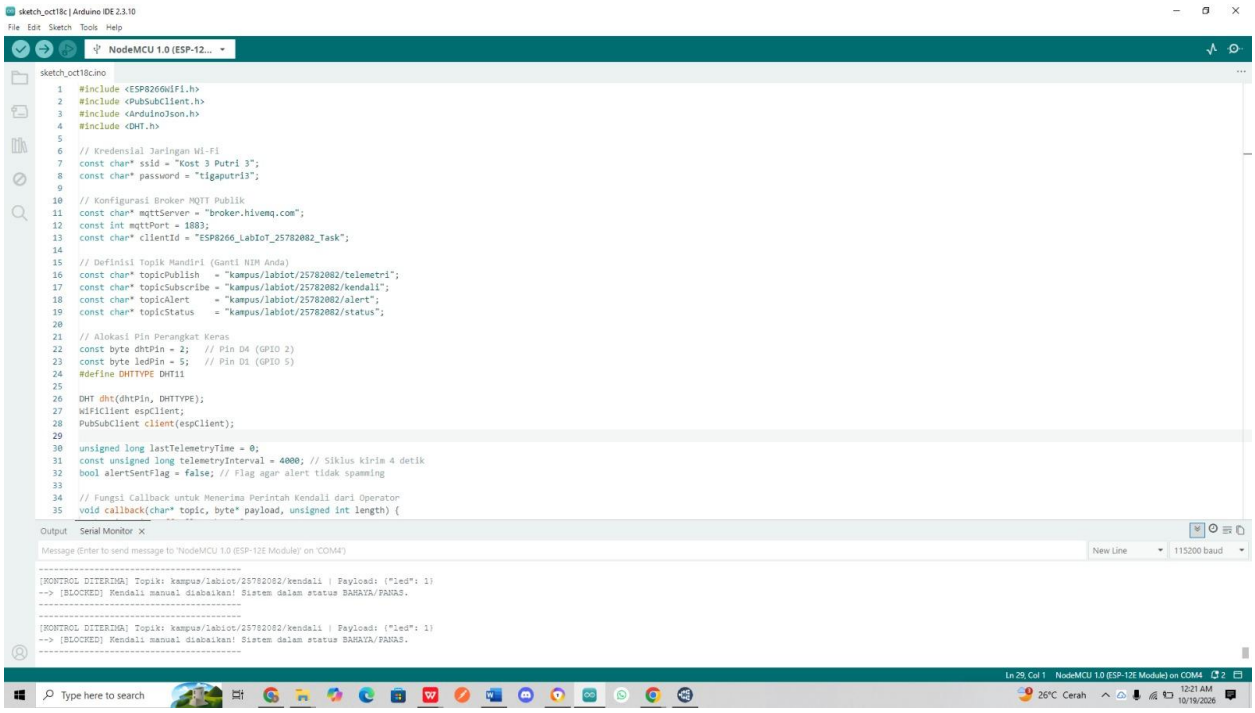
      // Kirim alert sekali saja saat pertama kali masuk kondisi panas (menggunakan flag)
      if (!alertSentFlag) {
        StaticJsonDocument<256> alertDoc;
        alertDoc["status"] = "BAHAYA";
        alertDoc["parameter"] = "OVERHEAT";
        alertDoc["nilai_suhu"] = serialized(String(t, 2));

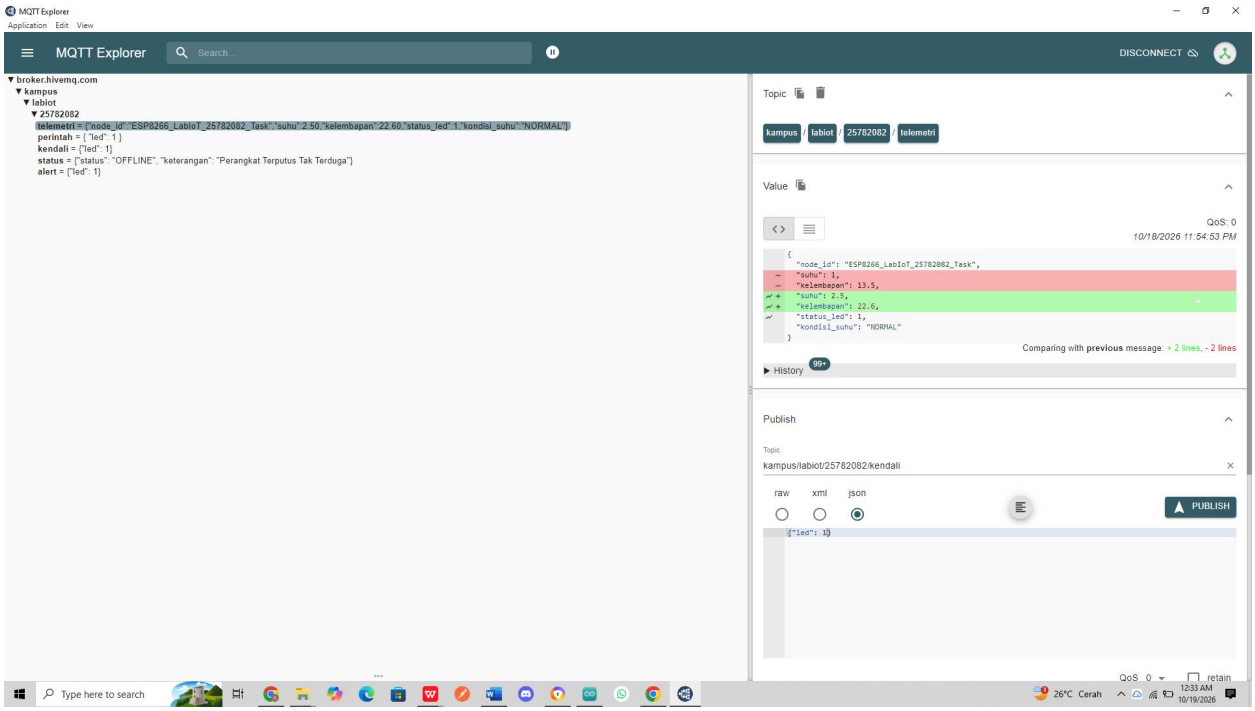
        char alertBuffer[256];
        serializeJson(alertDoc, alertBuffer);
        client.publish(topicAlert, alertBuffer, true);

        Serial.printf("[WARNING ALERT] Suhu Overheat! Terkirim ke %s\n", topicAlert);
        alertSentFlag = true; // Set flag aktif
      }
    } else {
      // Reset flag jika suhu sudah kembali normal
      if (alertSentFlag) {
        alertSentFlag = false;
        Serial.println("[INFO] Suhu kembali normal. Sistem pulih.");
      }
    }
  }
}
```


sketch_oct18c | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct18c.ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
// Kredensial Jaringan Wi-Fi
const char* ssid = "Kost 3 Putri 3";
const char* password = "tigaputri3";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782082_Task";
// Definisi Topik Mandiri (Ganti NIM Anda)
const char* topicPublish = "kampus/labiot/25782082/telemetri";
const char* topicSubscribe = "kampus/labiot/25782082/kendali";
const char* topicAlert = "kampus/labiot/25782082/alert";
const char* topicStatus = "kampus/labiot/25782082/status";
// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik
bool alertSentFlag = false; // Flag agar alert tidak spamming
// Fungsi Callback untuk Menerima Perintah Kendali dari Operator
void callback(char* topic, byte* payload, unsigned int length) {
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Full","suhu":1.00,"kelembapan":20.90,"status_led":0}
.....
Wi-Fi Terkoneksi!
IP Address: 192.168.18.72
Menghubungkan kembali ke Broker...Sukses Terhubung!
Subskripsi Topik: kampus/labiot/25782082/kendali
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782082_Task","suhu":1.20,"kelembapan":8.10,"status_led":0,"kondisi_suhu":"NORMAL"}
Ln 8, Col 35 NodeMCU 1.0 (ESP-12E Module) on COM4
Type here to search
26°C Cerah
11:39 PM
10/18/2026


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782082
telemetri = {"node_id":"ESP8266_LabIoT_25782082_Task","suhu":1.60,"kelembapan":24.30,"status_led":0,"kondisi_suhu":"NORMAL"}
perintah = { "led": 1 }
kendali = {"led": 0}
status = {"status": "ONLINE", "keterangan": "Perangkat Beroperasi Normal"}
Value
QoS: 0
10/18/2026 11:42:21 PM
{
"node_id": "ESP8266_LabIoT_25782082_Task",
"suhu": 1.6,
"kelembapan": 24.4,
"kelembapan": 24.3,
"status_led": 0,
"kondisi_suhu": "NORMAL"
}
Comparing with previous message: + 1 line, - 1 line
History
99+
Publish
Topic
kampus/labiot/25782082/kendali
raw
xml
json
PUBLISH
{"led": 0}
QoS 0
retain
History
kampus/labiot/25782082/kendali
{"led": 0}
kampus/labiot/25782082/kendali
Type here to search
25°C Cerah
11:42 PM
10/18/2026





C. Hasil Pengujian & Analisis Tugas

- Telemetri Rutin: Paket data suhu dan kelembapan dikirimkan secara stabil setiap 4 detik lengkap dengan klasifikasi status "kondisi_suhu" ("NORMAL" atau "PANAS").
- Otomasi Keselamatan (*Overheat Safety*): Ketika sensor mendeteksi suhu melampaui ambang batas 31.0 C, sistem lokal langsung memicu penyalaan LED indikator pada pin

D1 secara otomatis serta memublikasikan data peringatan ke topik kampus/labiot/25782082/alert.

- Kendali Manual (*Override*): Operator luar dapat mengambil alih kendali LED melalui topik kampus/labiot/25782082/kendali selama kondisi suhu berada dalam batas aman.
- Fitur LWT (*Last Will and Testament*): Saat kabel daya USB mikrokontroler dicabut secara mendadak, *broker* mendeteksi pemutusan tidak wajar dan otomatis mengubah status pada topik kampus/labiot/25782082/status menjadi OFFLINE.